

Annual Project: Truss Optimization

Objectives :

- Building a finite element solver for 2D bar truss
- Preparation of the solver for the use of optimization tools
- Perform parametric analysis and optimization of the truss

In this project, we are interested in the realization of an optimization of 2D bar trusses. We will focus on the realization of a versatile finite element bar solver capable of

- taking into account a given mesh (points and connectivity)
- building the operators of the finite element problem (stiffness matrix and nodal force vectors)
- solving the finite element problem taking into account the boundary conditions
- calculating the stresses and strains within the bars

A complete parametric constrained optimization will then be performed using this solver.

1 Project organization

The project is organized following several elements:

- Each problem (described in the following section) will be solved by a two-person team.
- The achievement of the project is divided into 4 supervised timeslots and additional autonomous work at home.

At the end the working phase, each group will have to produce:

- A synthetic report (between 5-8 pages) presenting the approach followed, the main developments and the results obtained (including figures and tables, but none of the produced code). It is strongly recommended to use $\text{\LaTeX}$ for the production of this report. The report must be given to the supervisors using Moodle's deposit form before **January 16th, 2026 (11pm)**.
- The developed code (notebook and/or **Python only** scripts). The code must be given to the supervisors using Moodle's deposit form before **January 16th, 2026 (11pm)**.
- A brief oral presentation (20 min per group) summarizing the work carried out and the results obtained. The presentation will take place during the last supervised session (**January 20th, 2026**). It will be composed of a 7 min oral contribution per member of the group followed by a 6 min questions/answers session.

Discussions about performance (number of iterations, number of calls, quality of the optimum, computation time, etc.) must be considered as expected parts of the report and the oral presentation.

NB: Plagiarism, use of AI tools and lack of referencing of external sources (code, documents, websites, etc.) will be sanctioned according to the rules in force at the CNAM.

2 Problems statement

In the case of this project, we are interested in optimization of 2D bar trusses. In order to do this, the reference problem will be firstly considered to validate the whole proposed optimization process. Afterwards, a complete truss problem will be considered to perform the optimization (see sub-section 2).

For all these problems, the following characteristics will be considered (See Table 1).

Material	Steel
Young modulus E	200 GPa
Poisson's coefficient ν	0,266
Density ρ	7860 kg/m ³
Yield limit σ_l	250 MPa
Length L	1 m

Table 1: Data of the problem

2.1 Reference problem

As a reference problem, the truss geometry in figure 1 studied in tutorial class and for which the analytical solution is known will be considered.

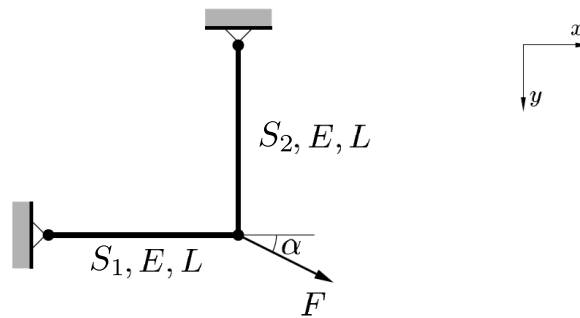


Figure 1: Truss geometry studied in tutorial class ($\alpha = 30^\circ$)

Additional, optimization problem studied in tutorial class could be considered.

2.2 Multi-bars truss problem

We are now interested in optimizing the truss geometry shown in figure 2. The optimization problem consists in minimizing the mass of the truss under the constraint of mechanical resistance of the bars (yield limit and buckling criteria). The considered parameters will be the areas of the cross-sections of the bars. We are still considering a truss of bars that only work in tension-compression. The truss is fixed (but could be extended to a larger one as a bonus of this project) but boundary conditions and loading differ for each two-person team. The different configurations are given in table 2.

Project	Fixed nodes on $\vec{x}$	Fixed nodes on $\vec{y}$	Load application node(s)	$\vec{x}$ load component	$\vec{y}$ load component
1	0,3,6	0,3,6	2, 5, 8	3000 N	-6000 N
2	0,2,3,5,6,8	0,2	7	0 N	-6000 N
3	0,2,3,5,6,8	0,2	1	-1000 N	-6000 N
4	0,3,6	0,2,3,5,6,8	1,4	-6000 N	6000 N
5	0,1,2,3,5,6,8	0,1,2	7,4	1000 N	6000 N
6	0,2,3,5,6,8	0,2,3,5,6,8	1,4	-6000 N	3000 N

Table 2: Configurations for each team

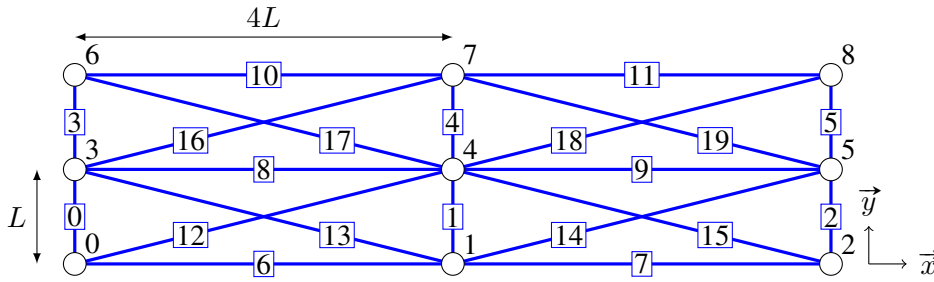


Figure 2: Multi-bar truss geometry

3 Preliminary work

- 1 Remind the formulation of the problem of minimizing the mass of the truss under the constraint of mechanical resistance of the bars (yield limit and buckling criteria).
- 2 Remind the analytical solution of the problem for the reference problem(s).
- 3 Adapt the formulation to the affected multi-bar truss problem.

4 Construction of the finite element solver

Instructions :

- Open the file `libTruss.py`: this file contains routines for visualizing the truss, preparing the data and achieving the post-processing. We will use in particular the routine `plotResults` which allows to visualize the mesh and display a result. The file must remain in the same folder as the working file. It is then called via the command `import libTruss`.
- The file `libTruss_dev.py` will contain your developments throughout the project. It contains empty routines to be filled in during the different steps of the project.
- Open the notebook `2bars_FE.ipynb`. The development steps of the solver will have to be carried out there.
- The developments made will be the subject of a synthetic report that does not show the produced code but the approach.
- If needed, for practical reasons (debug for example), the files provided are also available in `.py` format. You can use VScode to open them.

4.1 Stiffness matrix of an element bar

The considered problem only contains bar elements. These are considered in bi-rotated assembly and therefore only work in tension-compression.

By considering the bar j of length l_j (cf. figure 3), having 4 degrees of freedom (2 per node), it is recalled that the elementary stiffness matrix of the bar is written:

$$\mathbf{K}_j = \frac{Ex_j}{l_j} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -sc & -s^2 & cs & s^2 \end{bmatrix} \quad (1)$$

with $c = \cos \theta_j$, $s = \sin \theta_j$, E the Young's modulus, x_j the area of the cross-section j , l_j the length of the bar and θ_j the angle of the j th bar with respect to the horizontal. The following degrees of freedom vector is

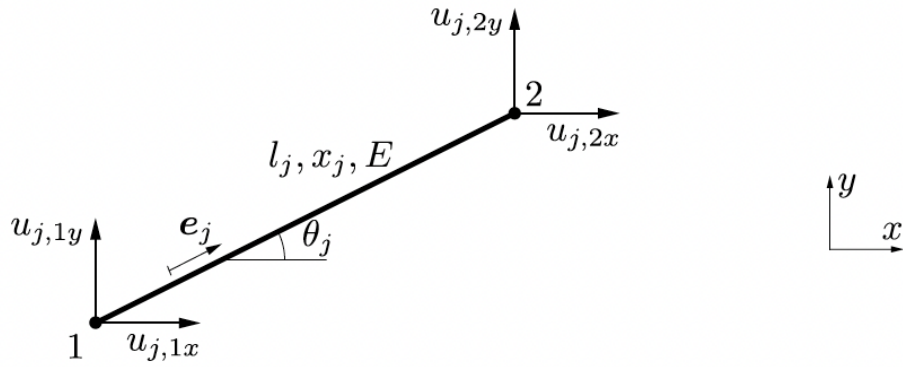


Figure 3: Geometry of a bar element

associated with this elementary stiffness matrix:

$$\mathbf{u}_j = \begin{bmatrix} u_{j,1x} \\ u_{j,1y} \\ u_{j,2x} \\ u_{j,2y} \end{bmatrix} \quad (2)$$

1 Fill in the routine `getElemBarStiffnessMatrix` to calculate the elementary stiffness matrix of a bar. It will take as arguments the coordinates of the 2 end nodes of the bar, the Young's modulus and the section of the bar.

2 Check your code by applying the routine to the considered truss.

4.2 Assembly of the global stiffness matrix

In practice, the complete finite element problem is based on the assembly of a global stiffness matrix integrating (1) all the elementary stiffness matrices of the truss bars and (2) all the degrees of freedom of all the nodes. Thus, a global vector of nodal displacements $\mathbf{u}$ is constructed in the form:

$$\mathbf{u} = \begin{bmatrix} u_{1x} \\ u_{1y} \\ u_{2x} \\ u_{2y} \\ \vdots \\ u_{nx} \\ u_{ny} \end{bmatrix} \quad (3)$$

This vector is thus of size $2n$ with n the number of nodes of the truss. Note the difference in notation between the degrees of freedom of the elementary bar and those of the global truss. Indeed, a node can be attached to several bars and the associated degrees of freedom are then shared. They only appear once in the vector $\mathbf{u}$.

Each elementary matrix can then be written in relation to the global degrees of freedom. Thus, for example, take the bar j whose elementary degrees of freedom of nodes 1 and 2 are such that they are associated with the global nodes k and m , we have

$$u_{j,1x} = u_{kx}, \quad u_{j,1y} = u_{ky}, \quad u_{j,2x} = u_{mx}, \quad u_{j,2y} = u_{my} \quad (4)$$

The initial elementary matrix $\mathbf{K}_j$ is then rewritten in the form:

$$\mathbf{K}_j^g = \frac{Ex_j}{l_j} \begin{bmatrix} 0 & \cdots & 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots \\ 0 & \cdots & c^2 & cs & \cdots & -c^2 & -cs & \cdots & 0 \\ 0 & \cdots & cs & s^2 & \cdots & -cs & -s^2 & \cdots & 0 \\ \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots \\ 0 & \cdots & -c^2 & -cs & \cdots & c^2 & cs & \cdots & 0 \\ 0 & \cdots & -sc & -s^2 & \cdots & cs & s^2 & \cdots & 0 \\ \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix} \quad \text{avec} \quad \mathbf{u} = \begin{bmatrix} u_{1x} \\ \vdots \\ u_{kx} \\ u_{ky} \\ \vdots \\ u_{mx} \\ u_{my} \\ \vdots \\ u_{ny} \end{bmatrix} \quad (5)$$

The matrix $\mathbf{K}_j^g$ and the vector $\mathbf{u}$ are then respectively of size $2n \times 2n$ and $2n \times 1$.

The global stiffness matrix $\mathbf{K}$ is then constructed by assembling the elementary stiffness matrices $\mathbf{K}_j^g$ of the N_e bars as follows:

$$\mathbf{K} = \sum_{j=1}^{N_e} \mathbf{K}_j^g \quad (6)$$

- 1 Fill in the routine `getGlobalStiffnessMatrix` to calculate the global stiffness matrix of the truss. It will take as arguments the coordinates of the nodes, the connectivity matrix, the Young's modulus and the section of the bars. It will use the previous function `getElemBarStiffnessMatrix`.
- 2 Check your code by applying the routine to the considered truss. We will ensure, among other things, the symmetry of the obtained matrix.

4.3 Building the force vector

The force vector $\mathbf{F}$ is constructed by taking into account the nodal forces applied to the nodes of the truss. These forces are defined by a vector of size $2n \times 1$ with n the number of nodes of the truss. Thus, for a global node i of coordinates (x_i, y_i) and subjected to a force $\vec{F}_i$ of coordinates (F_{ix}, F_{iy}) , we have

$$\mathbf{f} = \begin{bmatrix} 0 \\ \vdots \\ F_{ix} \\ F_{iy} \\ \vdots \\ 0 \end{bmatrix} \quad (7)$$

- 1 Fill in the routine `getGlobalForceVector` to calculate the global force vector of the truss. It will take as argument information related to the loads (node number and force vectors on each of them).
- 2 Check your implementation by applying the routine to the considered truss.

4.4 Solving the finite element problem

The finite element problem resolution consists in finding the value of the degrees of freedom of the problem (here the displacements u_{ix} and u_{iy} for $i \in \llbracket 0, n \rrbracket$). This problem is obtained by studying the equilibrium of the truss and is written as follows:

$$\mathbf{K}\mathbf{u} = \mathbf{f} \quad (8)$$

- 1 Explain why this problem cannot be solved directly by inverting the global stiffness matrix $\mathbf{K}$. To do this, you can use the `linalg` library of `numpy`.

We propose to rewrite the problem (8) by distinguishing the **known** and **unknown** degrees of freedom. We note $\mathbf{u}_k$ the vector of **known** degrees of freedom (**known** displacements) and $\mathbf{u}_u$ the vector of **unknown** degrees of freedom. We then have:

$$\begin{bmatrix} \mathbf{K}_{kk} & \mathbf{K}_{ku} \\ \mathbf{K}_{uk} & \mathbf{K}_{uu} \end{bmatrix} \begin{bmatrix} \mathbf{u}_k \\ \mathbf{u}_u \end{bmatrix} = \begin{bmatrix} \mathbf{f}_k \\ \mathbf{f}_u \end{bmatrix} \quad (9)$$

This can be rewritten in the form of two matrix equations:

$$\begin{cases} \mathbf{K}_{kk}\mathbf{u}_k + \mathbf{K}_{ku}\mathbf{u}_u = \mathbf{f}_k \\ \mathbf{K}_{uk}\mathbf{u}_k + \mathbf{K}_{uu}\mathbf{u}_u = \mathbf{f}_u \end{cases} \quad (10)$$

- 1 The vectors $\mathbf{u}_k$ and $\mathbf{f}_u$ being known, rewrite the problem to determine $\mathbf{u}_u$.
- 2 Fill in the routine `solveFEPproblem` to solve the finite element problem. It will take as argument the global stiffness matrix, the global force vector and information related to the boundary conditions in displacement ($\mathbf{u}_k$).
 - `solve` routine of the `linalg` library of `numpy` will be used.
 - We will check that the matrix problem can be solved (invertible matrix).
 - Finally, we will rebuild the solution vector of the problem $\mathbf{u} = \begin{bmatrix} \mathbf{u}_k \\ \mathbf{u}_u \end{bmatrix}$.
- 3 Check the consistency of the obtained results (direction and amplitude of the obtained displacements). The `plotResults` routine of the `libTruss` library provided can be used.

4.5 Calculation of stresses and strains

The constraints and deformations can be calculated for each bar of the truss based on the nodal displacements obtained. This is a so-called *post-processing* step. The deformation is calculated for a bar j , of length l_j and section x_j according to the formula:

$$\varepsilon_j = \frac{(\vec{u}_{j,2} - \vec{u}_{j,1}) \cdot \vec{e}_j}{l_j} \quad (11)$$

where $\vec{u}_{j,1}$, $\vec{u}_{j,2}$ and $\vec{e}_j$ are respectively the displacement vectors of the nodes 1 and 2 at the ends of the bar j and the director vector of the bar j (cf. figure 3). The stress is then given by Hooke's law:

$$\sigma_j = E\varepsilon_j \quad (12)$$

- 1 Fill in the routines `getElemStressStrain` and `getStressStrain` to calculate the stress and strain of a bar and the set of stresses and strains of the truss.
- 2 Check the consistency of the obtained results and validate them from the analytical results.

4.6 Parametric analysis

We are now looking to perform a parametric study of the reference problem (see sub-section 2.1) using the developed finite element solver. The aim is to study the influence of the section areas of the bars on the mass of the truss and the stresses in the bars.

- 1 Fill in the routine `parametricFE` so that it only takes as input parameters the coordinates of the nodes, the connectivity table and the areas of the sections of the bars (We will ensure that the areas are the first arguments of the list) and that it returns the mass of the truss (routine `getMassTruss` to be completed) and the stresses in the bars. We will use the previously developed routines as much as possible.
- 2 Use this routine to perform the parametric analysis of the optimization problem. Thus, we will study the evolution of the objective function and the constraint function(s) as a function of the design parameters (areas of the sections of the bars).

- 3 Plot the 3D surfaces and iso-surfaces using `matplotlib`.
- 4 Check the obtained results by comparing them to those obtained analytically.

4.7 Parametric gradients calculation (Bonus section)

In the context of parametric optimization, it may be useful (especially for optimizers) to calculate the gradients of the objective and constraint functions, i.e. the derivatives of these functions with respect to the design parameters (areas of the sections of the bars).

The parametric finite element problem is written in the form:

$$\mathbf{K}(\mathbf{x})\mathbf{u}(\mathbf{x}) = \mathbf{f} \quad (13)$$

where $\mathbf{x}$ is the vector of design parameters such that $\mathbf{x} = [x_1 \ \cdots \ x_{N_e}]^T$ with x_i the area of the section of the bar i .

By differentiating the equation (13) with respect to $\mathbf{x}$, we obtain:

$$\frac{d\mathbf{K}(\mathbf{x})}{d\mathbf{x}}\mathbf{u}(\mathbf{x}) + \mathbf{K}(\mathbf{x})\frac{d\mathbf{u}(\mathbf{x})}{d\mathbf{x}} = \mathbf{0} \quad (14)$$

We know that $\mathbf{u}(\mathbf{x})$ has already been calculated during the finite element resolution and taking into account the boundary conditions,

- 1 Formulate the problem to calculate $\frac{d\mathbf{u}(\mathbf{x})}{d\mathbf{x}}$.
- 2 Fill in the routine `getGlobalStiffnessMatrixDerivative` to calculate the derivative of the global stiffness matrix with respect to the design parameters. It will take as arguments the coordinates of the nodes, the connectivity matrix, the Young's modulus and the section of the bars.
- 3 Fill in the routine `getGlobalForceVectorDerivative` to calculate the derivative of the global force vector with respect to the design parameters. It will take as argument information related to the loads (node number and force vectors on each of them).
- 4 Check the consistency of the obtained results by using for instance order 1 or 2 finite differences.

5 Performing the optimization

In order to perform the optimization of the truss, we will use the `scipy.optimize` library. This library provides a set of optimizers to solve optimization problems under constraints. The file `example-optimize.ipynb` also presents an implementation of the library on a simple optimization problem. We will preferably use the COBYLA and COBYQA algorithms (other algorithms could be considered from other Python's libraries).

- 1 After completing your script by adapting the optimization problem to the case studied, perform the optimization of the 2-bar truss.
- 2 Present the results obtained and compare them to those obtained analytically.
- 3 Discuss the performance of the algorithm (number of iterations and number of calls to the solver among others when the gradients are provided or not).

Bonus switch to global optimizers (differential evolution, genetic algorithms, etc.) and compare the results obtained.

NB: if necessary and in order to facilitate the convergence of the optimization algorithm, we can proceed to

- the normalization of the objective and constraint functions (so that they have the same orders of magnitude).

- apply a change of variables (by taking for example the inverse of the design parameters).
- adjust the bounds of the design parameters.

These steps generally improve the convergence performance of the optimizers.

6 Multi-bar truss optimization

We will now consider the multi-bar truss problem presented in sub-section 2.2.

- 1** *Adapt your previous code to be able to manage a larger truss, additional constraints (especially the buckling criteria) and perform the optimization of this truss.*